01 - Hybrid - Pitt

In this notebook, we will try a hybrid (joint) architecture with Pitt set. This model contains:

- Tfidf (as in classical model)
- Bert embeddings (using sentence-transformer library)
- Bert summzarizer model
- Logistic regression classifier

In [1]:
```
# !pip install sentence-transformers
# !pip install bert-extractive-summarizer
```

In [2]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import precision_recall_fscore_support, f1_score, classificati
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sentence_transformers import SentenceTransformer
from summarizer import Summarizer
```

In [3]:
```
# some main options about using Tfidf, sentence embeddings and summary
train_ratio = 1.0             # the fraction of training set to use. Set to 1.0 for
use_summary = True            # set to False to skip summarization text
embed_model_name = "LaBSe"    # LaBSe model provides good performance and efficienc
concat_vector = True          # combine Tfidf + embeddings. Set to False to use Tfi
use_embeddings_only = False   # set to True to use Bert embedding only, ignoring Tf
tuning_score = "f1-score"     # can be recall / precision / f1-score. Use for hyper
```

In [4]:
```
# load Pitt set
train_fname = "../../data/pitt/pitt_train_txt.csv"
dev_fname = "../../data/pitt/pitt_dev_txt.csv"

train_df = pd.read_csv(train_fname)
dev_df = pd.read_csv(dev_fname)

print("Train len =", len(train_df))
print("Dev df =", len(dev_df))
```

```
Train len = 337
Dev df = 115
```

In [5]:
```
# in low-resource setting, maybe it's good to use a smaller subset of train data
# to prevent overfitting on train set

train_df = train_df.sample(frac=train_ratio, random_state=42)
train_df.reset_index(inplace=True)
print(len(train_df))
```

```
337
```

In [6]:
```
train_df.head()
```

Out[6]:

	index	id	file	target	text	split_name
0	46	120	120-1.cha	AD	okay .\nokay .\nokay .\nwell ‡ the boy is <rea...	train
1	163	579	579-0.cha	AD	woman dish\nboy climbin(g) up to get some cook...	train
2	57	168	168-0.cha	AD	uh the &+b boy's &+f gonna fall off of the &-...	train
3	25	65	065-0.cha	AD	um a &+g a girl is putting her finger up to h...	train
4	113	334	334-0.cha	AD	stool <is being>\nthe little girl is &-uh crit...	train

In [7]:
```
dev_df.head()
```

Out[7]:

	id	file	target	text	split_name
0	16	016-0.cha	AD	guy or a boy reaching for a cookie jar &-uh ti...	dev
1	16	016-1.cha	AD	uh the water's spilling over .\ndrying the dis...	dev
2	16	016-3.cha	AD	a boy is reaching for the cookie jar and tripp...	dev
3	16	016-4.cha	AD	well the little girl wants the cookie that the...	dev
4	33	033-0.cha	AD	okay .\napparently a mother and a daughter and...	dev

Pre-processing

We will lowercase the text for better efficiency

In [8]:
```
train_df["text_lower"] = train_df["text"].apply(lambda text: text.lower().strip())
dev_df["text_lower"] = dev_df["text"].apply(lambda text: text.lower().strip())
```

In [9]:
```
train_df.head()
```

Out[9]:

	index	id	file	target	text	split_name	text_lower
0	46	120	120-1.cha	AD	okay .\nokay .\nokay .\nwell ‡ the boy is <rea...	train	okay .\nokay .\nokay .\nwell ‡ the boy is <rea...
1	163	579	579-0.cha	AD	woman dish\nboy climbin(g) up to get some cook...	train	woman dish\nboy climbin(g) up to get some cook...
2	57	168	168-0.cha	AD	uh the &+b boy's &+f gonna fall off of the &-...	train	uh the &+b boy's &+f gonna fall off of the &-...
3	25	65	065-0.cha	AD	um a &+g a girl is putting her finger up to h...	train	um a &+g a girl is putting her finger up to h...
4	113	334	334-0.cha	AD	stool <is being>\nthe little girl is &-uh crit...	train	stool <is being>\nthe little girl is &-uh crit...

In [10]:
```
dev_df.head()
```

Out[10]:

	id	file	target	text	split_name	text_lower
0	16	016-0.cha	AD	guy or a boy reaching for a cookie jar &-uh ti...	dev	guy or a boy reaching for a cookie jar &-uh ti...
1	16	016-1.cha	AD	uh the water's spilling over .\ndrying the dis...	dev	uh the water's spilling over .\ndrying the dis...
2	16	016-3.cha	AD	a boy is reaching for the cookie jar and tripp...	dev	a boy is reaching for the cookie jar and tripp...
3	16	016-4.cha	AD	well the little girl wants the cookie that the...	dev	well the little girl wants the cookie that the...
4	33	033-0.cha	AD	okay .\napparently a mother and a daughter and...	dev	okay .\napparently a mother and a daughter and...

Apply summarizer model to shorten the text

In [11]:
```
# initialize summarization model
sum_model = Summarizer()
```

In [12]:
```
import warnings
# hide warnings
warnings.filterwarnings('ignore')
```

In [13]:
```
# test text summarization
s = """
"now wait a minute .
everything that's going on in the picture just by looking at it ?
```

```
well, she's wiping dishes .
and the boy is gettin(g) cookies out_of a jar .
and he's on a stool .
and there's cups over here .
and the water's going on the floor .
and the housewife, the mother, is wiping the dishes .
and she's standing by the window .
and &-uh she's lettin(g) the water run out_of the faucet onto the
and the  &+k the ladder is fallin(g) over when he's gettin(g) the
what else you want ?",AD
355-1,"well ‡ the boy is falling .
he's trying to get something  a cookie jar .
and <the stool is>  (.) he's gonna be on the floor in a minute
and &-uh he is tryin(g) to get a cookie for &+h this gal .
and she's over here &-uh +...
oh ‡ &+sh she's not payin(g) any attention .
and she didn't turn off the water .
and the water is drippin(g) all over the floor .
do you see that ?
and she's trying to &-uh +...
=laughs she's in a mess .
she's got <in a>  &+f water all over the floor .
and she's standing in the water .
and &-uh she has an apron on .
and she's tryin(g) to wipe the plates dry .
and &-uh (...) +..."
"""
summary = sum_model(s)
print(summary)
```

```
everything that's going on in the picture just by looking at it ? he's gonna be on t
he floor in a minute
and &-uh he is tryin(g) to get a cookie for &+h this gal . and the water is drippin
(g) all over the floor .
```

```
  File "C:\Users\Minh UBC\miniconda3\Lib\site-packages\joblib\externals\loky\backend
\context.py", line 282, in _count_physical_cores
    raise ValueError(f"found {cpu_count_physical} physical cores < 1")
```

In [14]:
```
if use_summary:
    summ_f = lambda text: f"{text.lower()}. Summary: {sum_model(text).lower()}"
else:
    summ_f = lambda text: f"{text.lower()}" # use it to skip summarization
```

In [15]:
```
# apply summary to dataframes
print("Performing summarization")
print("train set...")

train_df["text_lower"] = train_df["text"].apply(summ_f)

print("dev set...")
dev_df["text_lower"] = dev_df["text"].apply(summ_f)

print("Completed")
```

```
Performing summarization
train set...
dev set...
Completed
```

Build Tfidf Vectorizer

We can use Tfidf Vectorizer to convert text to a fixed-length vector

In [16]:
```
vectorizer = TfidfVectorizer()

corpus = train_df["text_lower"]
print(len(corpus))
print("Building Tfidf Vectorizer...")
X_train_tfidf = vectorizer.fit_transform(corpus)
print("Completed")
```

```
337
Building Tfidf Vectorizer...
Completed
```

In [17]:
```
print(X_train_tfidf.shape)
```

```
(337, 1438)
```

Create transformer-based sentence embeddings

In [18]:
```
# initialize sentence embeddings model
vectorizer_embed = SentenceTransformer(embed_model_name)
```

In [19]:
```
corpus = train_df["text_lower"]
print("Creating text embeddings...")
X_train_embed = vectorizer_embed.encode(corpus)
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [20]:
```
print(type(X_train_embed))
print(X_train_embed.shape)
```

```
<class 'numpy.ndarray'>
(337, 768)
```

In [21]:
```
# double check Tfidf vector
print(type(X_train_tfidf))
print(X_train_tfidf.shape)
```

```
<class 'scipy.sparse._csr.csr_matrix'>
(337, 1438)
```

Concate 2 vectors (Tfidf + sentence embeddings)

In [22]:
```
# test vector concatenation
v1 = [[2, 4, 6],
```

```
      [1, 3, 5]
     ]
v2 = [[8, 10],
      [7, 9]
     ]
v3 = np.concatenate((v1, v2), axis=1) # concate on horizontal axis
print(v3) # should be [2, 4, 6, 8, 10], [1, 3, 5, 7, 9]
assert np.allclose(v3,np.array([[2, 4, 6, 8, 10], [1, 3, 5, 7, 9]]))
print("Passed")
```

```
[[ 2  4  6  8 10]
 [ 1  3  5  7  9]]
Passed
```

In [23]:
```
def combine_vectors(tfidf_vec, embed_vec):
    """
        combine vectors using global settings (concat_vecor and use_embeddings_only
    """
    if concat_vector:
        result = np.concatenate((tfidf_vec.toarray(), embed_vec), axis=1)
    elif use_embeddings_only:
        result = embed_vec
    else: # use tfidf only
        result = tfidf_vec

    return result
```

In [24]:
```
X_train = combine_vectors(X_train_tfidf, X_train_embed)
print(X_train.shape)
```

```
(337, 2206)
```

Train Logistic regression using combined vector

We can now train a simple logistic regression classifier using processed X_train

In [25]:
```
y_train = train_df["target"]

print("Training logistic classifier...")

clf = LogisticRegression(random_state=42,
                         class_weight= "balanced"
                        ).fit(X_train, y_train)

print("Completed")
```

```
Training logistic classifier...
Completed
```

In [26]:
```
# getting score on train set
train_score = clf.score(X_train, y_train)
print(f"Train accuracy = {train_score*100:.2f} %")
```

```
Train accuracy = 86.94 %
```

In [27]:
```
# get precision / recall / f1
y_pred = clf.predict(X_train)
print(classification_report(y_train, y_pred, target_names=["AD", "CN"]))
```

```
              precision    recall  f1-score   support

          AD       0.92      0.84      0.88       187
          CN       0.82      0.91      0.86       150

    accuracy                           0.87       337
   macro avg       0.87      0.87      0.87       337
weighted avg       0.87      0.87      0.87       337
```

Now we can evaluate on dev set

In [28]:
```
X_dev_tfidf = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]
print("X_dev shape =", X_dev_tfidf.shape)
print("y_dev len =", len(y_dev))
```

```
X_dev shape = (115, 1438)
y_dev len = 115
```

In [29]:
```
# create X_dev embeddings
print("Creating text embeddings...")
X_dev_embed = vectorizer_embed.encode(dev_df["text_lower"])
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [30]:
```
print(type(X_dev_embed))
print(X_dev_embed.shape)
```

```
<class 'numpy.ndarray'>
(115, 768)
```

In [31]:
```
# double check Tfidf vector
print(type(X_dev_tfidf))
print(X_dev_tfidf.shape)
```

```
<class 'scipy.sparse._csr.csr_matrix'>
(115, 1438)
```

In [32]:
```
X_dev = combine_vectors(X_dev_tfidf, X_dev_embed)
print(X_dev.shape)
```

```
(115, 2206)
```

In [33]:
```
dev_score = clf.score(X_dev, y_dev)
print(f"Dev accuracy = {dev_score*100:.2f} %")
```

```
Dev accuracy = 78.26 %
```

In [34]:
```
y_pred = clf.predict(X_dev)
print(classification_report(y_dev, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.83      0.77      0.80        64
          CN       0.73      0.80      0.77        51

    accuracy                           0.78       115
   macro avg       0.78      0.78      0.78       115
weighted avg       0.79      0.78      0.78       115
```

In [35]:
```
# extract AD tuning score
print("Tuning score =", tuning_score)
classification_report(y_dev, y_pred, output_dict=True)["AD"][tuning_score]
```

```
Tuning score = f1-score
```

Out[35]:
```
0.7967479674796748
```

Hyperparameter tuning

We can adjust a few parameters on the Tfidf vectorizer and class_weight in the Logistic regression

In [36]:
```
# following are sets of parameters for tuning
tfidf_params_max_features = [None, 1000, 5000, 10000]
tfidf_ngrams = [(1, 1), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4)]

lr_class_weights = [None,
                    "balanced",
                    {"AD":0.65, "CN":0.35},
                    {"AD":0.4, "CN":0.6},
                    {"AD":0.45, "CN":0.55},
                    {"AD":0.5, "CN":0.5},
                    {"AD":0.55, "CN":0.45},
                    {"AD":0.6, "CN":0.4},
                    {"AD":0.65, "CN":0.35}
                   ]

best_params = dict()

best_dev_score = 0      # overall accuracy score
best_avg_dev_score = 0  # average between accuracy and tuning score (recall / f1 /

print(f"Hyper parameter tuning on accuracy and AD {tuning_score}...")
for max_feat in tfidf_params_max_features:
    for ngram in tfidf_ngrams:
        for class_weights in lr_class_weights:
            print(f"Params, max_feat = {max_feat}, ngram = {ngram}, class_weights =

            vectorizer = TfidfVectorizer(max_features=max_feat, ngram_range=ngram)

            X_train_tfidf = vectorizer.fit_transform(train_df["text_lower"])
            X_train = combine_vectors(X_train_tfidf, X_train_embed)

            # create logistic regression classifier
```

```
            clf = LogisticRegression(random_state=42,
                                     class_weight= class_weights
                                    ).fit(X_train, y_train)

            # getting score on train set
            train_score = clf.score(X_train, y_train)
            print(f"Train accuracy = {train_score*100:.2f} %")

            y_pred_train = clf.predict(X_train)
            train_tune_score = classification_report(y_train, y_pred_train, output_
            print(f"Train {tuning_score} = {train_tune_score*100:.2f} %")

            # getting score on dev set
            X_dev_tfidf = vectorizer.transform(dev_df["text_lower"])
            X_dev = combine_vectors(X_dev_tfidf, X_dev_embed)
            y_dev = dev_df["target"]

            dev_score = clf.score(X_dev, y_dev)
            print(f"Dev accuracy = {dev_score*100:.2f} %")

            # extract AD recall rate
            y_pred_dev = clf.predict(X_dev)
            dev_tuning_score = classification_report(y_dev, y_pred_dev, output_dict
            print(f"Dev {tuning_score} = {dev_tuning_score*100:.2f} %")

            dev_avg_score = (dev_score + dev_tuning_score) / 2

            if dev_avg_score > best_avg_dev_score:
                print("**** New best average dev score found =", dev_avg_score)

                best_dev_score = dev_score
                best_avg_dev_score = dev_avg_score
                best_params["max_feat"] = max_feat
                best_params["class_weights"] = class_weights
                best_params["ngram"] = ngram

            print("-------")


print("Completed")
print("Best dev score =", best_dev_score)
print("Best average dev score =", best_avg_dev_score)

print("Best params =", best_params)
```

```
Hyper parameter tuning on accuracy and AD f1-score...
Params, max_feat = None, ngram = (1, 1), class_weights = None
Train accuracy = 88.72 %
Train f1-score = 90.00 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
**** New best average dev score found = 0.7853940217391304
-------
Params, max_feat = None, ngram = (1, 1), class_weights = balanced
Train accuracy = 86.94 %
Train f1-score = 87.71 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
**** New best average dev score found = 0.7896783315659244
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.38 %
Train f1-score = 83.53 %
Dev accuracy = 77.39 %
Dev f1-score = 77.19 %
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 83.98 %
Train f1-score = 84.92 %
Dev accuracy = 79.13 %
Dev f1-score = 80.33 %
**** New best average dev score found = 0.7972915181753385
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 83.98 %
Train f1-score = 85.86 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 85.46 %
Train f1-score = 88.02 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
**** New best average dev score found = 0.797319385420072
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.47 %
Dev accuracy = 74.78 %
Dev f1-score = 79.72 %
-------
Params, max_feat = None, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
```

```
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = None
Train accuracy = 93.77 %
Train f1-score = 94.52 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = balanced
Train accuracy = 91.39 %
Train f1-score = 92.05 %
Dev accuracy = 80.00 %
Dev f1-score = 80.99 %
**** New best average dev score found = 0.8049586776859505
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 65.22 %
Dev f1-score = 75.61 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.46 %
Train f1-score = 85.55 %
Dev accuracy = 80.00 %
Dev f1-score = 79.65 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.83 %
Train f1-score = 88.64 %
Dev accuracy = 80.00 %
Dev f1-score = 80.99 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.61 %
Train f1-score = 90.96 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.94 %
Train f1-score = 89.37 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.64 %
Train f1-score = 83.86 %
Dev accuracy = 73.91 %
Dev f1-score = 79.45 %
-------
Params, max_feat = None, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 65.22 %
```

```
Dev f1-score = 75.61 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = None
Train accuracy = 94.66 %
Train f1-score = 95.31 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = balanced
Train accuracy = 93.47 %
Train f1-score = 94.02 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 66.77 %
Train f1-score = 76.95 %
Dev accuracy = 62.61 %
Dev f1-score = 73.94 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.35 %
Train f1-score = 86.39 %
Dev accuracy = 79.13 %
Dev f1-score = 78.57 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 88.72 %
Train f1-score = 89.50 %
Dev accuracy = 80.00 %
Dev f1-score = 80.67 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.39 %
Train f1-score = 92.58 %
Dev accuracy = 76.52 %
Dev f1-score = 78.74 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.94 %
Train f1-score = 89.47 %
Dev accuracy = 75.65 %
Dev f1-score = 79.10 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.64 %
Train f1-score = 83.86 %
Dev accuracy = 73.04 %
Dev f1-score = 78.91 %
-------
Params, max_feat = None, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 66.77 %
Train f1-score = 76.95 %
Dev accuracy = 62.61 %
Dev f1-score = 73.94 %
-------
```

```
Params, max_feat = None, ngram = (1, 4), class_weights = None
Train accuracy = 94.96 %
Train f1-score = 95.56 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = balanced
Train accuracy = 94.07 %
Train f1-score = 94.57 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.28 %
Train f1-score = 76.17 %
Dev accuracy = 62.61 %
Dev f1-score = 73.94 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.94 %
Train f1-score = 86.98 %
Dev accuracy = 79.13 %
Dev f1-score = 78.57 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 89.61 %
Train f1-score = 90.36 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 92.28 %
Train f1-score = 93.30 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.54 %
Train f1-score = 89.90 %
Dev accuracy = 75.65 %
Dev f1-score = 79.41 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.34 %
Train f1-score = 83.67 %
Dev accuracy = 71.30 %
Dev f1-score = 77.85 %
-------
Params, max_feat = None, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.28 %
Train f1-score = 76.17 %
Dev accuracy = 62.61 %
Dev f1-score = 73.94 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = None
Train accuracy = 95.55 %
```

```
Train f1-score = 96.10 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = balanced
Train accuracy = 94.36 %
Train f1-score = 94.82 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.58 %
Train f1-score = 76.33 %
Dev accuracy = 60.00 %
Dev f1-score = 72.29 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 87.24 %
Train f1-score = 87.32 %
Dev accuracy = 80.87 %
Dev f1-score = 80.36 %
**** New best average dev score found = 0.8061335403726708
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 89.91 %
Train f1-score = 90.66 %
Dev accuracy = 77.39 %
Dev f1-score = 78.33 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 92.28 %
Train f1-score = 93.30 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.24 %
Train f1-score = 89.69 %
Dev accuracy = 75.65 %
Dev f1-score = 79.41 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 77.45 %
Train f1-score = 83.11 %
Dev accuracy = 67.83 %
Dev f1-score = 75.82 %
-------
Params, max_feat = None, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.58 %
Train f1-score = 76.33 %
Dev accuracy = 60.00 %
Dev f1-score = 72.29 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = None
Train accuracy = 95.55 %
Train f1-score = 96.08 %
```

```
Dev accuracy = 78.26 %
Dev f1-score = 80.00 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = balanced
Train accuracy = 94.66 %
Train f1-score = 95.11 %
Dev accuracy = 78.26 %
Dev f1-score = 78.99 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.28 %
Train f1-score = 76.17 %
Dev accuracy = 61.74 %
Dev f1-score = 73.81 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.94 %
Train f1-score = 86.98 %
Dev accuracy = 80.87 %
Dev f1-score = 80.36 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 90.21 %
Train f1-score = 90.96 %
Dev accuracy = 77.39 %
Dev f1-score = 78.33 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.99 %
Train f1-score = 93.06 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.35 %
Train f1-score = 89.05 %
Dev accuracy = 75.65 %
Dev f1-score = 79.41 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 77.74 %
Train f1-score = 83.30 %
Dev accuracy = 66.09 %
Dev f1-score = 74.84 %
-------
Params, max_feat = None, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 65.28 %
Train f1-score = 76.17 %
Dev accuracy = 61.74 %
Dev f1-score = 73.81 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = None
Train accuracy = 88.43 %
Train f1-score = 89.71 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
```

```
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = balanced
Train accuracy = 86.94 %
Train f1-score = 87.71 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.38 %
Train f1-score = 83.53 %
Dev accuracy = 76.52 %
Dev f1-score = 76.11 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 83.98 %
Train f1-score = 85.00 %
Dev accuracy = 78.26 %
Dev f1-score = 79.34 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 84.27 %
Train f1-score = 86.16 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 84.87 %
Train f1-score = 87.53 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.53 %
Train f1-score = 84.21 %
Dev accuracy = 74.78 %
Dev f1-score = 79.72 %
-------
Params, max_feat = 1000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = None
Train accuracy = 90.21 %
Train f1-score = 91.20 %
Dev accuracy = 77.39 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = balanced
```

```
Train accuracy = 87.24 %
Train f1-score = 88.02 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.68 %
Train f1-score = 83.97 %
Dev accuracy = 77.39 %
Dev f1-score = 76.79 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 85.46 %
Train f1-score = 86.35 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 85.46 %
Train f1-score = 87.14 %
Dev accuracy = 77.39 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.05 %
Train f1-score = 88.51 %
Dev accuracy = 76.52 %
Dev f1-score = 79.70 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.47 %
Dev accuracy = 74.78 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 1000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = None
Train accuracy = 89.61 %
Train f1-score = 90.67 %
Dev accuracy = 75.65 %
Dev f1-score = 78.46 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = balanced
Train accuracy = 87.83 %
Train f1-score = 88.58 %
```

```
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 68.70 %
Dev f1-score = 77.22 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.98 %
Train f1-score = 84.39 %
Dev accuracy = 79.13 %
Dev f1-score = 78.95 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 86.05 %
Train f1-score = 86.91 %
Dev accuracy = 75.65 %
Dev f1-score = 77.78 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 86.05 %
Train f1-score = 87.66 %
Dev accuracy = 75.65 %
Dev f1-score = 78.46 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.65 %
Train f1-score = 89.00 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.47 %
Dev accuracy = 77.39 %
Dev f1-score = 81.94 %
-------
Params, max_feat = 1000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 68.70 %
Dev f1-score = 77.22 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = None
Train accuracy = 89.61 %
Train f1-score = 90.62 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = balanced
Train accuracy = 87.83 %
Train f1-score = 88.58 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
```

```
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 68.70 %
Dev f1-score = 77.22 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.68 %
Train f1-score = 84.06 %
Dev accuracy = 79.13 %
Dev f1-score = 78.95 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 85.46 %
Train f1-score = 86.43 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 86.35 %
Train f1-score = 87.83 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.65 %
Train f1-score = 89.00 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 80.12 %
Train f1-score = 84.74 %
Dev accuracy = 75.65 %
Dev f1-score = 80.56 %
-------
Params, max_feat = 1000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 68.70 %
Dev f1-score = 77.22 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = None
Train accuracy = 90.80 %
Train f1-score = 91.73 %
Dev accuracy = 76.52 %
Dev f1-score = 78.74 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = balanced
Train accuracy = 90.50 %
Train f1-score = 91.21 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
```

```
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 66.96 %
Dev f1-score = 76.25 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.05 %
Train f1-score = 86.38 %
Dev accuracy = 78.26 %
Dev f1-score = 77.88 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.24 %
Train f1-score = 88.09 %
Dev accuracy = 78.26 %
Dev f1-score = 79.34 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.02 %
Train f1-score = 90.34 %
Dev accuracy = 77.39 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 85.76 %
Train f1-score = 88.41 %
Dev accuracy = 79.13 %
Dev f1-score = 82.09 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 80.12 %
Train f1-score = 84.81 %
Dev accuracy = 73.91 %
Dev f1-score = 79.73 %
-------
Params, max_feat = 1000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.92 %
Train f1-score = 79.24 %
Dev accuracy = 66.96 %
Dev f1-score = 76.25 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = None
Train accuracy = 91.39 %
Train f1-score = 92.23 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = balanced
Train accuracy = 90.50 %
Train f1-score = 91.21 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.33 %
Train f1-score = 78.90 %
```

```
Dev accuracy = 66.96 %
Dev f1-score = 76.54 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.35 %
Train f1-score = 86.63 %
Dev accuracy = 79.13 %
Dev f1-score = 78.95 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.83 %
Train f1-score = 88.64 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 88.13 %
Train f1-score = 89.53 %
Dev accuracy = 77.39 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.35 %
Train f1-score = 88.78 %
Dev accuracy = 78.26 %
Dev f1-score = 81.48 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 80.12 %
Train f1-score = 84.81 %
Dev accuracy = 74.78 %
Dev f1-score = 80.54 %
-------
Params, max_feat = 1000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.33 %
Train f1-score = 78.90 %
Dev accuracy = 66.96 %
Dev f1-score = 76.54 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = None
Train accuracy = 88.72 %
Train f1-score = 90.00 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = balanced
Train accuracy = 86.94 %
Train f1-score = 87.71 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
```

```
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.38 %
Train f1-score = 83.53 %
Dev accuracy = 77.39 %
Dev f1-score = 77.19 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 83.98 %
Train f1-score = 84.92 %
Dev accuracy = 79.13 %
Dev f1-score = 80.33 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 83.98 %
Train f1-score = 85.86 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 85.46 %
Train f1-score = 88.02 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.47 %
Dev accuracy = 74.78 %
Dev f1-score = 79.72 %
-------
Params, max_feat = 5000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = None
Train accuracy = 93.77 %
Train f1-score = 94.52 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = balanced
Train accuracy = 91.10 %
Train f1-score = 91.76 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.33 %
Train f1-score = 78.90 %
Dev accuracy = 66.96 %
Dev f1-score = 76.54 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.4, 'CN': 0.6}
```

```
Train accuracy = 84.87 %
Train f1-score = 85.04 %
Dev accuracy = 78.26 %
Dev f1-score = 77.48 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.83 %
Train f1-score = 88.64 %
Dev accuracy = 79.13 %
Dev f1-score = 80.33 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.91 %
Train f1-score = 91.24 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.65 %
Train f1-score = 89.10 %
Dev accuracy = 77.39 %
Dev f1-score = 80.60 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.93 %
Train f1-score = 84.04 %
Dev accuracy = 73.91 %
Dev f1-score = 79.45 %
-------
Params, max_feat = 5000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.33 %
Train f1-score = 78.90 %
Dev accuracy = 66.96 %
Dev f1-score = 76.54 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = None
Train accuracy = 93.47 %
Train f1-score = 94.24 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = balanced
Train accuracy = 91.39 %
Train f1-score = 92.05 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 69.44 %
Train f1-score = 78.41 %
Dev accuracy = 65.22 %
Dev f1-score = 75.00 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.16 %
Train f1-score = 85.29 %
```

```
Dev accuracy = 80.00 %
Dev f1-score = 79.65 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.24 %
Train f1-score = 88.09 %
Dev accuracy = 77.39 %
Dev f1-score = 79.03 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.91 %
Train f1-score = 91.33 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.94 %
Train f1-score = 89.37 %
Dev accuracy = 79.13 %
Dev f1-score = 82.09 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 80.12 %
Train f1-score = 84.81 %
Dev accuracy = 73.04 %
Dev f1-score = 78.91 %
-------
Params, max_feat = 5000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 69.44 %
Train f1-score = 78.41 %
Dev accuracy = 65.22 %
Dev f1-score = 75.00 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = None
Train accuracy = 93.77 %
Train f1-score = 94.52 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = balanced
Train accuracy = 91.69 %
Train f1-score = 92.27 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.03 %
Train f1-score = 78.74 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.16 %
Train f1-score = 85.29 %
Dev accuracy = 79.13 %
Dev f1-score = 78.57 %
```

```
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.54 %
Train f1-score = 88.40 %
Dev accuracy = 77.39 %
Dev f1-score = 79.03 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.91 %
Train f1-score = 91.24 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.65 %
Train f1-score = 89.10 %
Dev accuracy = 79.13 %
Dev f1-score = 82.09 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.62 %
Dev accuracy = 73.91 %
Dev f1-score = 79.73 %
-------
Params, max_feat = 5000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 70.03 %
Train f1-score = 78.74 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = None
Train accuracy = 94.07 %
Train f1-score = 94.76 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = balanced
Train accuracy = 93.18 %
Train f1-score = 93.73 %
Dev accuracy = 78.26 %
Dev f1-score = 79.34 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 61.74 %
Dev f1-score = 73.49 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 86.35 %
Train f1-score = 86.55 %
Dev accuracy = 80.00 %
Dev f1-score = 79.28 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
```

```
Train accuracy = 88.43 %
Train f1-score = 89.26 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.69 %
Train f1-score = 92.78 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.54 %
Train f1-score = 89.90 %
Dev accuracy = 76.52 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.53 %
Train f1-score = 84.42 %
Dev accuracy = 70.43 %
Dev f1-score = 77.33 %
-------
Params, max_feat = 5000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 61.74 %
Dev f1-score = 73.49 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = None
Train accuracy = 94.07 %
Train f1-score = 94.76 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = balanced
Train accuracy = 93.18 %
Train f1-score = 93.73 %
Dev accuracy = 79.13 %
Dev f1-score = 80.33 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.84 %
Train f1-score = 78.08 %
Dev accuracy = 62.61 %
Dev f1-score = 74.56 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 87.24 %
Train f1-score = 87.61 %
Dev accuracy = 79.13 %
Dev f1-score = 78.57 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 88.72 %
Train f1-score = 89.56 %
```

```
Dev accuracy = 77.39 %
Dev f1-score = 78.69 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.69 %
Train f1-score = 92.78 %
Dev accuracy = 78.26 %
Dev f1-score = 80.62 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.54 %
Train f1-score = 89.90 %
Dev accuracy = 77.39 %
Dev f1-score = 80.60 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.53 %
Train f1-score = 84.42 %
Dev accuracy = 71.30 %
Dev f1-score = 78.15 %
-------
Params, max_feat = 5000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.84 %
Train f1-score = 78.08 %
Dev accuracy = 62.61 %
Dev f1-score = 74.56 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = None
Train accuracy = 88.72 %
Train f1-score = 90.00 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = balanced
Train accuracy = 86.94 %
Train f1-score = 87.71 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 83.38 %
Train f1-score = 83.53 %
Dev accuracy = 77.39 %
Dev f1-score = 77.19 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 83.98 %
Train f1-score = 84.92 %
Dev accuracy = 79.13 %
Dev f1-score = 80.33 %
```

```
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 83.98 %
Train f1-score = 85.86 %
Dev accuracy = 76.52 %
Dev f1-score = 79.07 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 85.46 %
Train f1-score = 88.02 %
Dev accuracy = 78.26 %
Dev f1-score = 81.20 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.82 %
Train f1-score = 84.47 %
Dev accuracy = 74.78 %
Dev f1-score = 79.72 %
-------
Params, max_feat = 10000, ngram = (1, 1), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 71.22 %
Train f1-score = 79.41 %
Dev accuracy = 69.57 %
Dev f1-score = 77.71 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = None
Train accuracy = 93.77 %
Train f1-score = 94.52 %
Dev accuracy = 78.26 %
Dev f1-score = 80.31 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = balanced
Train accuracy = 91.39 %
Train f1-score = 92.05 %
Dev accuracy = 80.87 %
Dev f1-score = 81.67 %
**** New best average dev score found = 0.8126811594202898
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.46 %
Train f1-score = 85.55 %
Dev accuracy = 78.26 %
Dev f1-score = 77.48 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 87.83 %
Train f1-score = 88.64 %
Dev accuracy = 80.00 %
Dev f1-score = 80.99 %
-------
```

```
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 89.61 %
Train f1-score = 90.96 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.94 %
Train f1-score = 89.37 %
Dev accuracy = 77.39 %
Dev f1-score = 80.60 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.93 %
Train f1-score = 84.04 %
Dev accuracy = 73.91 %
Dev f1-score = 79.45 %
-------
Params, max_feat = 10000, ngram = (1, 2), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.55 %
Train f1-score = 77.92 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = None
Train accuracy = 94.36 %
Train f1-score = 95.06 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = balanced
Train accuracy = 92.28 %
Train f1-score = 92.93 %
Dev accuracy = 76.52 %
Dev f1-score = 78.05 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.84 %
Train f1-score = 78.08 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.46 %
Train f1-score = 85.55 %
Dev accuracy = 80.00 %
Dev f1-score = 79.65 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 88.13 %
Train f1-score = 88.95 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.10 %
```

```
Train f1-score = 92.31 %
Dev accuracy = 76.52 %
Dev f1-score = 78.74 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 86.65 %
Train f1-score = 89.16 %
Dev accuracy = 77.39 %
Dev f1-score = 80.60 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.53 %
Train f1-score = 84.42 %
Dev accuracy = 73.04 %
Dev f1-score = 78.91 %
-------
Params, max_feat = 10000, ngram = (1, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.84 %
Train f1-score = 78.08 %
Dev accuracy = 64.35 %
Dev f1-score = 74.85 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = None
Train accuracy = 94.07 %
Train f1-score = 94.79 %
Dev accuracy = 76.52 %
Dev f1-score = 78.74 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = balanced
Train accuracy = 92.88 %
Train f1-score = 93.44 %
Dev accuracy = 77.39 %
Dev f1-score = 79.03 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.25 %
Train f1-score = 77.75 %
Dev accuracy = 63.48 %
Dev f1-score = 74.39 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 85.46 %
Train f1-score = 85.63 %
Dev accuracy = 78.26 %
Dev f1-score = 77.48 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 88.43 %
Train f1-score = 89.20 %
Dev accuracy = 78.26 %
Dev f1-score = 79.67 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 90.80 %
Train f1-score = 91.99 %
Dev accuracy = 77.39 %
```

```
Dev f1-score = 79.69 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.54 %
Train f1-score = 89.86 %
Dev accuracy = 76.52 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 79.23 %
Train f1-score = 84.23 %
Dev accuracy = 72.17 %
Dev f1-score = 78.38 %
-------
Params, max_feat = 10000, ngram = (1, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 68.25 %
Train f1-score = 77.75 %
Dev accuracy = 63.48 %
Dev f1-score = 74.39 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = None
Train accuracy = 95.25 %
Train f1-score = 95.83 %
Dev accuracy = 77.39 %
Dev f1-score = 79.37 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = balanced
Train accuracy = 94.07 %
Train f1-score = 94.59 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 66.47 %
Train f1-score = 76.80 %
Dev accuracy = 60.87 %
Dev f1-score = 73.05 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 87.24 %
Train f1-score = 87.39 %
Dev accuracy = 80.87 %
Dev f1-score = 80.36 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 89.61 %
Train f1-score = 90.41 %
Dev accuracy = 78.26 %
Dev f1-score = 79.34 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.99 %
Train f1-score = 93.06 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
```

```
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.54 %
Train f1-score = 89.90 %
Dev accuracy = 75.65 %
Dev f1-score = 79.41 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.93 %
Train f1-score = 84.04 %
Dev accuracy = 69.57 %
Dev f1-score = 76.82 %
-------
Params, max_feat = 10000, ngram = (2, 3), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 66.47 %
Train f1-score = 76.80 %
Dev accuracy = 60.87 %
Dev f1-score = 73.05 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = None
Train accuracy = 94.96 %
Train f1-score = 95.58 %
Dev accuracy = 76.52 %
Dev f1-score = 78.40 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = balanced
Train accuracy = 93.77 %
Train f1-score = 94.31 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 67.06 %
Train f1-score = 77.11 %
Dev accuracy = 62.61 %
Dev f1-score = 74.56 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.4, 'CN': 0.6}
Train accuracy = 87.24 %
Train f1-score = 87.46 %
Dev accuracy = 80.00 %
Dev f1-score = 79.28 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.45, 'CN': 0.55}
Train accuracy = 89.61 %
Train f1-score = 90.41 %
Dev accuracy = 79.13 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.5, 'CN': 0.5}
Train accuracy = 91.39 %
Train f1-score = 92.51 %
Dev accuracy = 77.39 %
Dev f1-score = 79.69 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.55, 'CN': 0.45}
Train accuracy = 87.24 %
```

```
Train f1-score = 89.69 %
Dev accuracy = 76.52 %
Dev f1-score = 80.00 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.6, 'CN': 0.4}
Train accuracy = 78.64 %
Train f1-score = 83.86 %
Dev accuracy = 69.57 %
Dev f1-score = 76.82 %
-------
Params, max_feat = 10000, ngram = (2, 4), class_weights = {'AD': 0.65, 'CN': 0.35}
Train accuracy = 67.06 %
Train f1-score = 77.11 %
Dev accuracy = 62.61 %
Dev f1-score = 74.56 %
-------
Completed
Best dev score = 0.808695652173913
Best average dev score = 0.8126811594202898
Best params = {'max_feat': 10000, 'class_weights': 'balanced', 'ngram': (1, 2)}
```

Create tfidf vectorizer and logistic regression classifier with new best params

In [37]:
```
vectorizer = TfidfVectorizer(max_features=best_params["max_feat"], ngram_range=best

X_train_tfidf = vectorizer.fit_transform(train_df["text_lower"])
X_train = combine_vectors(X_train_tfidf, X_train_embed)

clf = LogisticRegression(random_state =42,
                         class_weight = best_params["class_weights"]
                        ).fit(X_train, y_train)

# double check dev score
X_dev_tfidf = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]

X_dev = combine_vectors(X_dev_tfidf, X_dev_embed)


dev_score = clf.score(X_dev, y_dev)
print(f"Train accuracy = {clf.score(X_train, y_train)*100:.2f} %")
print(f"Dev accuracy = {dev_score*100:.2f} %")

assert dev_score == best_dev_score
```

```
Train accuracy = 91.39 %
Dev accuracy = 80.87 %
```

In [38]:
```
print(classification_report(y_train, clf.predict(X_train)))
```

```
                precision    recall  f1-score   support

            AD       0.94      0.90      0.92       187
            CN       0.88      0.93      0.91       150

      accuracy                           0.91       337
     macro avg       0.91      0.92      0.91       337
  weighted avg       0.92      0.91      0.91       337
```

In [39]:
```
print(classification_report(y_dev, clf.predict(X_dev)))
```

```
                precision    recall  f1-score   support

            AD       0.88      0.77      0.82        64
            CN       0.75      0.86      0.80        51

      accuracy                           0.81       115
     macro avg       0.81      0.81      0.81       115
  weighted avg       0.82      0.81      0.81       115
```

Process prediction for test set

In [40]:
```
test_fname = "../../data/pitt/pitt_test_txt.csv"

test_df = pd.read_csv(test_fname)
print("Test df =", len(test_df))
test_df["text_lower"] = test_df["text"].apply(lambda text: text.lower().strip())
test_df.head()
```

```
Test df = 97
```

Out[40]:

	id	file	target	text	split_name	text_lower
0	1	001-0.cha	AD	mhm .\nalright .\nthere's &-um a young boy tha...	test	mhm .\nalright .\nthere's &-um a young boy tha...
1	1	001-2.cha	AD	mhm .\nthere's a young boy &-uh going in a coo...	test	mhm .\nthere's a young boy &-uh going in a coo...
2	23	023-0.cha	AD	(..) well I see the mother .\nwhat's she doin(...	test	(..) well i see the mother .\nwhat's she doin(...
3	23	023-2.cha	AD	well you see <the little> the little boy stea...	test	well you see <the little> the little boy stea...
4	39	039-0.cha	AD	okay we see &-um first of all &-uh the sink is...	test	okay we see &-um first of all &-uh the sink is...

In [41]:
```
# add summarization
print("Performing summarization")
print("Test set...")

test_df["text_lower"] = test_df["text"].apply(summ_f)
```

```
print("Completed")
test_df.head()
```

```
Performing summarization
Test set...
Completed
```

Out[41]:

	id	file	target	text	split_name	text_lower
0	1	001-0.cha	AD	mhm .\nalright .\nthere's &-um a young boy tha...	test	mhm .\nalright .\nthere's &-um a young boy tha...
1	1	001-2.cha	AD	mhm .\nthere's a young boy &-uh going in a coo...	test	mhm .\nthere's a young boy &-uh going in a coo...
2	23	023-0.cha	AD	(..) well I see the mother .\nwhat's she doin(...	test	(..) well i see the mother .\nwhat's she doin(...
3	23	023-2.cha	AD	well you see <the little> the little boy stea...	test	well you see <the little> the little boy stea...
4	39	039-0.cha	AD	okay we see &-um first of all &-uh the sink is...	test	okay we see &-um first of all &-uh the sink is...

In [42]:
```
X_test_tfidf = vectorizer.transform(test_df["text_lower"])
```

In [43]:
```
# create X_test embeddings
print("Creating text embeddings...")
X_test_embed = vectorizer_embed.encode(test_df["text_lower"])
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [44]:
```
X_test = combine_vectors(X_test_tfidf, X_test_embed)

print(X_test.shape)
```

```
(97, 10768)
```

In [45]:
```
test_df["prediction"] = clf.predict(X_test)
test_df.head()
```

Out[45]:

	id	file	target	text	split_name	text_lower	prediction
0	1	001-0.cha	AD	mhm .\nalright .\nthere's &-um a young boy tha...	test	mhm .\nalright .\nthere's &-um a young boy tha...	AD
1	1	001-2.cha	AD	mhm .\nthere's a young boy &-uh going in a coo...	test	mhm .\nthere's a young boy &-uh going in a coo...	AD
2	23	023-0.cha	AD	(..) well I see the mother .\nwhat's she doin(...	test	(..) well i see the mother .\nwhat's she doin(...	AD
3	23	023-2.cha	AD	well you see <the little> the little boy stea...	test	well you see <the little> the little boy stea...	AD
4	39	039-0.cha	AD	okay we see &-um first of all &-uh the sink is...	test	okay we see &-um first of all &-uh the sink is...	CN

In [46]:
```
correct_count = len(test_df.query("target == prediction"))
print(f"Correct count = {correct_count}, accuracy = {correct_count*100/len(test_df)
```

```
Correct count = 88, accuracy = 91 %
```

In [47]:
```
print(classification_report(test_df["target"], test_df["prediction"]))
```

```
              precision    recall  f1-score   support

          AD       0.93      0.91      0.92        55
          CN       0.88      0.90      0.89        42

    accuracy                           0.91        97
   macro avg       0.90      0.91      0.91        97
weighted avg       0.91      0.91      0.91        97
```